

INFORMATION TECHNOLOGY: PAPER I

Time: 3 hours

150 marks

PLEASE READ THE FOLLOWING INSTRUCTIONS CAREFULLY

1. This question paper consists of 15 pages. Please check that your question paper is complete.
2. This question paper is to be answered using object-oriented programming principles. Your program must make sensible use of methods and parameters.
3. This paper is divided into two sections. All candidates must answer both sections.
4. This paper is set in programming terms that are not specific to any particular programming language (Java/Delphi) or database (Access/MySQL/JavaDB).
5. Ensure that you answer the questions in the manner described because marks will be awarded for your solution according to the specifications that are given in the question.
6. Only answer what is asked for in each question. For example, if the question does not ask for data validation, no marks are awarded for it; therefore, no code needs to be written for data validation.
7. If you cannot get a section of code to work, comment it out so that it will not be executed, and you can continue with the examination. If possible, try to explain the error to aid the marker.
8. Your programs must be coded so they will work with any data, not just the sample data supplied or any data extracts that appear in the question paper. You are advised to look at the provided data files carefully.
9. Make sure that routines such as searches and sorts for arrays are developed from first principles and that you do not use the built-in features of a programming language for any of these routines.
10. All data structures must be defined and declared by you, the programmer. You may not use the interface's components to store and retrieve data.
11. Read the whole question paper before you choose a data structure. You may find that there could be an alternative method of representing the data that will be more efficient, considering the questions asked in the paper.

12. You must save all your work regularly on the disk you have been given, or the disk space allocated to you for this examination. You should also create a backup of the original files before you start, in case your solution accidentally modifies the original version.
13. If your examination is interrupted by a technical problem such as a power failure, you will, when you resume writing, be given only the time remaining when the interruption began to complete your examination. No extra time will be given to catch up on work that was not saved.
14. Make sure your examination number appears as a comment in every program you code and on every printed page you hand in.
15. Check every printed page you print, making sure you submit the code for every question.
16. Check that your printout has correct indenting and formatting and has not truncated any text.
17. Check that each page is clearly legible with a font size of 12.
18. Print a code listing of all the programs/class files that you code. Printing must be done after the examination. You will be given half an hour to print after the examination. Your teacher will tell you what arrangements have been made to print your work.
19. You should have the following two folders (in bold) and files. These files are to be used as data for this examination. Note that the database files are in MS Access, JavaDB and MySQL formats and contain the same data in each file. You only need the file that matches the database application you will need for Section A. Ensure that you can open the files with the packages you will use to code your solutions to this examination.

Section A:

CGISoft_Access.mdb
CGISoft_JavaDB.sql
CGISoft_MySQL.sql
SQLAnswerSheet.rtf
SQLBrowser.exe

Section B:

Accounts.txt
Requests.txt

SCENARIO:**SECTION A SQL****QUESTION 1**

CGISoft sells software products to edit videos, photos and documents. They have a list of clients who have either bought software or shown interest in buying software. Each client is assigned a number from 1 to 3, representing the loyalty level they have achieved. The database listing the products, clients, and sales information is called **CGISoft**.

tblProducts contains the details of products.

FIELDS	DATA TYPE	DESCRIPTION
ProdID	INTEGER	A unique auto-numbered identification number for the product.
ProdName	TEXT	The name of the product.
Category	TEXT	The software category (Video Editor, Web Editor, Design & Publication, Image Editor) the product belongs to.
Fee	INTEGER	The cost to purchase the product.
Version	INTEGER	The version number of the product.

The first 10 records of tblProducts:

ProdID	ProdName	Category	Fee	Version
1	Video Magic	Video Editor	500	12
2	Video Magic Premium	Video Editor	615	13
3	Special Effects	Video Editor	325	15
4	Special Effects Premium	Video Editor	452	16
5	CGI Web Design	Web Editor	325	7
6	CGI Publications	Design & Publication	125	6
7	CGI Publications Deluxe	Design & Publication	153	7
8	Scalar Images	Image Editor	350	11
9	Scalar Images Premium	Image Editor	406	12
10	Super Notes	Design & Publication	150	18

tblClients contains the details of the clients.

FIELDS	DATA TYPE	DESCRIPTION
ClientID	INTEGER	A unique auto-numbered identification number for each client.
ClientName	TEXT	The full name of the client.
CellNo	TEXT	The cell phone number of the client.
Province	TEXT	The province in which the client lives.
Loyalty	INTEGER	The loyalty level of the client. An integer from 1 to 3.

The first 10 records of tblClients:

ClientID	ClientName	CellNo	Province	Loyalty
1	Khumo Khumalo	083-269-2524	Free State	2
2	Robin Mussett	082-745-7071	Gauteng	2
3	Arya Govender	066-160-2471	Eastern Cape	2
4	Thandiwe Hlophe	066-080-1487	Northern Cape	1
5	Deepak Singh	066-124-2349	Limpopo	1
6	Indra Ganguly	066-126-8930	Mpumalanga	1
7	Eric Sambaza	066-321-5923	Eastern Cape	1
8	Min Bai	083-607-6255	KwaZulu Natal	2
9	Mark Cimato	067-249-4979	Free State	1
10	Kobus Nel	087-880-9343	Northern Cape	2

tblSales contains the details of the sales of products to clients.

FIELDS	DATA TYPE	DESCRIPTION
ProdID	INTEGER	The ID of the product. This is a foreign key to tblProducts.
ClientID	INTEGER	The ID of the client. This is a foreign key to tblClients.
License	TEXT	The license number for the product.
SaleDate	DATE	The date the client bought the product.

The first 10 records of tblSales:

ProdID	ClientID	License	SaleDate
14	1	7646980	2024/09/19
4	2	2907254	2022/08/18
8	3	2386250	2023/07/09
14	4	1871512	2024/11/10
2	5	6714630	2022/01/10
3	6	7211144	2022/05/21
8	6	8157626	2023/07/10
5	11	6334116	2022/09/03
7	14	1133452	2023/01/27
11	14	5211395	2024/05/17

Note: Do not rename columns/use an alias, unless explicitly told to do so in a question.

- 1.1 Display all the details of the clients with a loyalty level of 3 who are not from Gauteng. The correct output is shown below:

ClientID	ClientName	CellNo	Province	Loyalty
11	James Harrison	087-295-4372	KwaZulu Natal	3
21	Nomalanga Dlamini	087-382-5934	North West	3

(5)

- 1.2 Display the product name and fee for each product with a name ending in either 'Premium' or 'Deluxe', as well as a fee over 400. The correct output is shown below:

ProdName	Fee
Video Magic Premium	615
Special Effects Premium	452
Scalar Images Premium	406
Photo Perfect Deluxe	416

(6)

- 1.3 Display the total number of sales that CGISoft has made. Name this field 'NumSales'. The correct output is shown below:

NumSales
27

(3)

- 1.4 CGISoft provides free product support from the date of purchase until the end of the next calendar year. For example, if a product was bought in April 2022, it would receive support until the end of December 2023.

Create a query to list the license numbers and sales dates for all products that currently receive free support. Sort the list by license number. You may NOT hardcode a specific year into your query. The correct output is shown below:

License	SaleDate
1871512	2024/11/10
3837789	2024/01/16
4398266	2024/07/20
5211395	2024/05/17
6789288	2024/04/03
7223099	2024/11/08
7646980	2024/09/19
9848551	2024/05/26

(5)

- 1.5 Display the client's name and the license of the product the client bought for all sales. The first 6 rows of output are shown below:

ClientName	License
Deepak Singh	6714630
Cheryl Smith	2316437
Indra Ganguly	7211144
Sihle Gule	7085762
Qian Wang	8477108
Robin Mussett	2907254

(4)

- 1.6 Display the product name and fee of all products with a fee greater than the average fee. The first 6 rows of output are shown below:

ProdName	Fee
Video Magic	500
Video Magic Premium	615
Special Effects	325
Special Effects Premium	452
CGI Web Design	325
Scalar Images	350

(5)

- 1.7 Display the province name and the total value of sales for each province. Name the second field 'ValueOfSales'. Only display provinces where the total value of sales is greater than 1000. The correct output is shown below:

Province	ValueOfSales
Gauteng	1080
KwaZulu Natal	1169
Northern Cape	2185

(7)

- 1.8 CGISoft wants to standardise its product names. Create a query to change the names of all products that end in 'Deluxe' to end in 'Premium' instead. For example, 'Photo Perfect Deluxe' should be renamed as 'Photo Perfect Premium'. You should create a single query to rename all the necessary products.

(8)

- 1.9 Create a query to add a row of sales data to tblSales. Use the following information:

Product ID: 14

Client ID: 25

License: Generate a random number from 1000000 to 2000000 (1 million to 2 million)

Date of sale: 2025/03/25

(7)

50 marks

SECTION B OBJECT-ORIENTED PROGRAMMING

CGISoft develops video editing software, which is available in a free and a premium version.

The free version:

- Can be used without a license.
- Must be activated every 6 months.

The premium version of the software requires a premium license to be purchased.

- The license applies to the number of devices the user/organisation pays for.
- The license must be activated every 6 months.
- Devices can be authorised up to the amount paid for by sending through the MAC address of the device.
- Devices must be validated by checking the last 2 digits of the license.

A premium license consists of 6 digits and must end with two digits as described below.

- The last 2 digits of the license are calculated as follows:
 - Adding the Unicode value of each character in the name of the user/organisation that purchased the license.
 - Determining the remainder when the sum is divided by 100.
 - If the remainder is greater than or equal to 10, the last 2 digits are the remainder.
 - If the remainder is less than 10, the last 2 digits are 0 followed by the remainder.
 - You can ignore the first 4 digits of the license.
- For example, if 'Rx' purchases a license:
 - R has a Unicode value of 82, x has a Unicode value of 120. Adding the Unicode value of each character gives $82 + 120 = 202$.
 - The remainder of $202/100$ is 2. Since 2 is less than 10, the last two digits would be 02.
 - So, if the license were 187302, it would be valid because the last two digits are 02.

Requests for either ACCESS or AUTHORISE will be represented by numbers as follows:

- 0 – ACCESS (when the program is run, an ACCESS request is made to check if the user or device is allowed to run the program).
- 1 – AUTHORISE (an AUTHORISE request activates the program for the free version, or it assigns a license to a device for the paid version).

Free Class

This class will represent the following basic details of a user/organisation with a free account.

- **name** – the full name of the user/organisation, stored as a string.
- **email** – the email of the user/organisation, stored as a string.
- **actDate** – the date the program was last activated, stored as a Date.

Premium Class

This class will represent the following basic details of a user/organisation with a premium account.

- **licNo** – the license number for the program, stored as a string.
- **qty** – the number of devices that the program can be authorised on, stored as an integer.
- **macArr** – the MAC addresses of the authorised devices, stored as an array of strings.

The details of the free accounts and the premium accounts are stored in the text file **Accounts.txt** in no specific order.

A line of the file representing a free account has the following format:

<name>#<email>#<actDate>

actDate is a date in the format **dd<space>MM<space>yyyy**

For example:

Thabisela Mthembu#thabisela@mthembu.co.za#15 06 2025

A line of the file representing a premium account has the following format:

<name>#<email>#<actDate>#<licNo>#<qty>

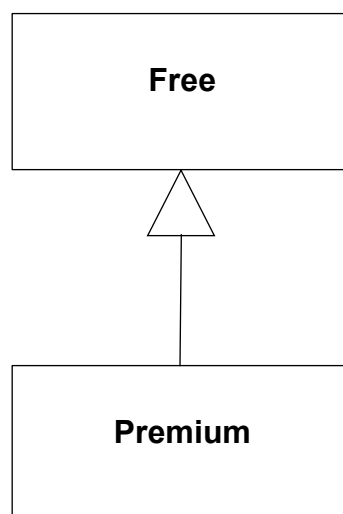
For example:

Xiaoyuan Hu#hu.x@gmail.com#14 07 2025#307767#1

The first 5 lines of data from the **Accounts.txt** text file are shown below:

```
Xiaoyuan Hu#hu.x@gmail.com#14 07 2025#307767#1
Digital Animatronics#procurement@diganim.com#23 02 2025#480690#4
Capital Pictures#cgi@capital.co.za#24 09 2024#991127#3
Thabisela Mthembu#thabisela@mthembu.co.za#15 06 2025
Umhlangu Holdings#enockm@umhlanguholdings.co.za#07 11 2024#884889#5
```

The relationship between the parent class **Free** and the child class **Premium** is shown in the diagram below.



Answer the questions that follow to create a program that will allow the user to activate the software, determine whether a device has access to the software and authorise devices. Your program does not need any sales or payment features.

QUESTION 2

Use the class diagram below to create a new class called **Free**. This class will be used to store the details of free accounts. The diagram below indicates the fields and methods that are required.

Free
Fields: - name : string - email : string - actDate : Date
Methods: + Constructor(inN : string, inE : string, inA : string) + getName(): string + setEmail(inE : string) + isActive() : boolean + authorise() : boolean + toString() : string

- 2.1 Create a new class named **Free** with the **name**, **email** and **actDate** fields, as shown in the class diagram. These fields must not be accessible outside the class. (3)
- 2.2 Code a parameterised constructor method that will accept parameters, as shown in the class diagram, and assign these values to the corresponding fields. **inN** corresponds to **name**. **inE** corresponds to **email**.

inA corresponds to **actDate**. Note that the **inA** parameter is a string and must be converted to a Date. (4)
- 2.3 Code an accessor method for the **name** field. (1)
- 2.4 Code a mutator method for the **email** field. (1)
- 2.5 Code a method called **isActive**. This method checks whether the program has been activated in the last six months. The method returns true if the current date is less than or equal to six months after the activation date. (3)
- 2.6 Code a method called **authorise**. This method should set the activation date to the current date and return true. (2)
- 2.7 Code a **toString** method that will return a string that combines the values of the **name**, **email** and **actDate** fields. The format of the string should be as follows:

name<space>**email**<space>**actDate**

For example:

Madelynn Smythe smythem@upe.ac.za 2025-07-12

(2)
[16]

QUESTION 3

Use the class diagram below to create a new class called **Premium**. This class will be used to store the details of premium accounts and is a child class of **Free**. The diagram below indicates the fields and methods that are required.

Premium
Fields: - licNo : string - qty : integer - macArr[] : string
Methods: + Constructor(inN : string, inE : string, inA : string inL : string, inQ : integer) + authorise(mac : string) : boolean + isAuthorised(mac : string) : boolean + isValid() : boolean + toString() : string

3.1 Create a new class named **Premium**, which is a child class of **Free**, with the **licNo**, **qty** and **macArr** fields as shown in the class diagram. These fields must not be accessible outside the class. (3)

3.2 Code a parameterised constructor method that will accept parameters, as shown in the class diagram. Initialise the fields of the **Free** parent class and **Premium** child class. **inL** corresponds to **licNo**. **inQ** corresponds to **qty**.

macArr must be initialised to have a size equal to the **qty** field. Initialise each value in **macArr** to the text 'Available'. (6)

3.3 Code a **toString** method that will override the parent class's **toString** method, add the **licNo** and **qty** fields to the parent's **toString** and return a string in the following format:

name<space>**email**<space>**actDate**<space>**licNo**<space>**qty**

For example:

Blue Sky Movies bsm@bsm.co.za 2025-04-11 300994 3 (3)

Note: The **authorise**, **isAuthorised** and **isValid** methods will be coded in Question 6.

[12]

QUESTION 4

- 4.1 Create a class called **RequestManager**. (1)
- 4.2 Create two fields for the **RequestManager** class as described below:
- An array called **accArr** to store up to 40 **Free** or **Premium** objects.
A counter called **accSize** to count the objects added to the array.
These fields should not be accessible from outside the class. (3)
- 4.3 Add two static constants to the class as described below. These constants must be accessible from outside the class:
- A static constant integer called **ACCESS** with a value of 0.
 - A static constant integer called **AUTHORISE** with a value of 1. (2)
- 4.4 Create a constructor method that will read the contents of the text file **Accounts.txt**, containing the information about **Free** and **Premium** accounts. The method should do the following:
- Read each line from the file **Accounts.txt** and extract the data.
 - Instantiate either a **Free** or **Premium** object based on the line's extracted data.
 - Add the **Free** or **Premium** object to the next available position in the array.
 - Increase the counter **accSize**. (11)
- 4.5 Code a method called **sortByName** to sort the objects in the array **accArr** in alphabetical order of the **name** field. (5)
- 4.6 Code a **toString** method to combine the values of each object in the array **accArr** into a string. The details of each object should be on a new line. Use the **toString** methods you created in Question 2 and Question 3. (4)
- [26]**

QUESTION 5

- 5.1 Create a text-based user interface called **RequestUI** that will allow simple input and output. (1)
- 5.2 Declare and instantiate a **RequestManager** object. (1)
- 5.3 Call the appropriate method in **RequestManager** to sort the data alphabetically by name. (1)
- 5.4 Display the data in the **accArr** array. The correct output is shown below. (1)

```
Blue Sky Movies bsm@bsm.co.za 2025-04-11 300994 3
Capital Pictures cgi@capital.co.za 2024-09-24 991127 3
Digital Animatronics procurement@diganim.com 2025-02-23 480690
4
John Prentice drifter91120@yahoo.com 2024-10-11 734057 2
Madelyyn Smythe smythem@upe.ac.za 2025-07-12
Simon Govender govender.s@safari.co.za 2024-08-28
Sizwe Ntini sizwel@gmail.com 2025-07-10 768576 1
Thabisela Mthembu thabisela@mthembu.co.za 2025-06-15
Tsiamo Kamangu Kamangu123@gmail.com 2025-02-21
Umhlangu Holdings enockm@umhlanguholdings.co.za 2024-11-07
884889 5
Xiaoyuan Hu hu.x@gmail.com 2025-07-14 307767 1
```

[4]

QUESTION 6

In the **Premium** class, add the following methods as shown in the class diagram found in Question 3 on page 10. Please code these methods AFTER the methods from Question 3.

NB: Remember that the **qty** field in the **Premium** class is the size of the **macArr** array.

6.1 Code a method called **isAuthorised** that accepts a string parameter representing the MAC address of a device.

- The method should search through **macArr** for a string with the value equal to the parameter.
- If the string is found, the method should return true.
- If the string is not found, the method should return false.

(4)

6.2 Code a method called **authorise** which accepts a string parameter representing the MAC address of a device.

- The method should search **macArr** for the first element with the value 'Available'.
- This element within the array should be set to the parameter MAC address, and the method should return true.
- The **actDate** should be updated to the current date.
- If no string with the value 'Available' is found, the method should return false.

(4)

6.3 Code a method called **isValid**. This method must check if the license number is valid. The steps are:

- Add the Unicode values of each character of the **name** field.
- Calculate the remainder when the sum of the Unicode values is divided by 100.
- Return true if the last 2 digits of the **licNo** are equal to the remainder.
- Return false otherwise.

(4)

[12]

QUESTION 7

In the **RequestManager** class:

- 7.1 Code a helper method called **processFree**, which accepts an integer parameter called **action** and a **Free** parameter called **account**.

This method should process **ACCESS** and **AUTHORISE** actions and return a string as follows:

ACCESS:

- An active account should return the message " has access to the free version."
- An inactive account should return the message " has an expired account."

AUTHORISE:

- The method must authorise the account and return the message " has been authorised for the free version."

(9)

- 7.2 Code a helper method called **processPremium** that accepts an integer parameter called **action**, a **Premium** parameter called **account**, and a string parameter called **mac**.

This method should process **ACCESS** and **AUTHORISE** actions as follows:

- Any account with an invalid license number should return the message " has an invalid license."

ACCESS:

- An active, authorised account should return the MAC address and the message " has access to the premium version."
- Otherwise, it should return the MAC address and the message " does not have access to the premium version."

AUTHORISE:

- Attempt to authorise the device.
- If the authorisation is successful, return the MAC address and the message " has been authorised for the premium version."
- Otherwise, return the MAC address and the message " is not authorised. Purchase another license."

(10)

- 7.3 The **ACCESS** and **AUTHORISE** requests are stored in a text file called **Requests.txt**.

Requests from a free version of the program are stored in the format:
<action>#<name>

Requests from a premium version of the program are stored in the format:
<action>#<name>#<mac>

Where:

- **action** is an integer representing an **ACCESS (0)** request or an **AUTHORISE (1)** request.
- **name** is a string representing the name of the account owner.
- **mac** is a string representing the MAC address of the device making the request.

The first three lines of **Requests.txt** are shown below:

```
1#Blue Sky Movies#F5:AE:8A:85:3A:C6
0#Blue Sky Movies#F5:AE:8A:85:3A:C6
0#Madelyyn Smythe
```

Code a method called **processRequests**. Read through the text file **Requests.txt**. Call the appropriate method to process each request based on the action value and return the feedback from all requests as a string. Each request's feedback should be on a new line. You should make use of the helper methods **processFree** and **processPremium**.

The format for feedback for a free version is:

name<space>**message**

The format for feedback for a premium version is:

name<space>**mac**<space>**message** (10)

In the **RequestUI** class:

7.4 Call the **processRequests** method and display the results. (1)

The correct output is shown below:

```
Blue Sky Movies F5:AE:8A:85:3A:C6 has been authorised for
the premium version.
Blue Sky Movies F5:AE:8A:85:3A:C6 has access to the
premium version.
Madelyyn Smythe has access to the free version.
Blue Sky Movies D5:1F:2B:8B:79:3C has been authorised for
the premium version.
Sizwe Ntini 56:E8:73:23:C8:37 does not have access to the
premium version.
Blue Sky Movies FC:F1:5B:40:6B:DC does not have access to
the premium version.
Blue Sky Movies A6:5F:88:79:C7:88 has been authorised for
the premium version.
Simon Govender has an expired account.
Simon Govender has been authorised for the free version.
Capital Pictures has an invalid license.
Blue Sky Movies FC:F1:5B:40:6B:DC is not authorised.
Purchase another license.
```

[30]

100 marks

Total: 150 marks